to save on shipment costs), that product's price may and will vary with time (so you can choose the cheapest brand for, say, brakes or suspensions) and even that some products might be unavailable sometimes.

All this complexity, however, can be captured in a heuristic function, a score that is computed keeping in mind all the nuances of your business: conceptually you will be able to handle that score in the same exact way as the simple inventory count, and so we can keep things simple in our example and just use that.

One way to handle these requirements could be using two different data structures: one for efficient search by name, for instance a hash table, and a priority queue to get the item which most urgently requires to be re-supplied.

We will see, in chapter 7, that sometimes combining two data structures for a goal is the best choice; this is not the time yet, for now you need to keep in mind, however, of the issues in coordinating such two containers, and also that you will likely need more than twice the memory.

Both considerations are kind of worrying: wouldn't it be nice if there was a single data structure that can handle both aspects natively and efficiently?

### 3.1.1 The Gist of the Solution

Let's be clear about what we are seeking here: it's not just a matter of optimizing all the operations on a container, like we discuss in appendix C: here each bit of data, each entry in the container, is made of two separate bits, and both can be "measured" in some way: there are the names of each product, which can be sorted alphabetically, and to each product we also